Return your solutions on Moodle before **Sunday 19.4. at 23:59**. Your solutions must include intermediate steps and explain your reasoning.

Exercise 7.1. Let g and h be differentiable functions. Write the derivatives of the following functions in terms of $g(x)$, $h(x)$, $g'(x)$, $h'(x)$, and elementary functions:

(a) $f(x) = g(x)^2 + h(x)$

(b) $f(x) = g(h(x))^2 + h(h(x))$

(c) $f(x) = g(x^3) + h(\cos(x))$

(d) $f(x) = h(g(x)^2) + \exp(h(x))$

Example: By the chain rule and $\frac{\mathrm{d}}{\mathrm{d}x}\sin(x) = \cos(x)$, the function $f(x) = 2\sin(g(x))$ has the derivative $f'(x) = 2\cos(g(x))g'(x)$.

Solution. Recall basic calculus results from *Differential Calculus* course such as the

$$\text{Chain rule:} \quad D(f \circ g)(x) = (Df \circ g)(x) \cdot Dg(x) \quad \text{for some differentiable functions } f, g,$$
$$\text{Linearity of differentiation:} \quad D[af(x) + bg(x)] = aDf(x) + bDg(x) \quad \text{for some } a, b \in \mathbb{R},$$

and known derivatives of elementary functions.

(a)

$$\begin{aligned}
Df(x) &= D[g(x)^2 + h(x)] \\
&= D[g(x)^2] + Dh(x) && \text{by linearity of differentiation} \\
&= D(t^2 \circ g)(x) + Dh(x) \\
&= (D[t^2] \circ g)(x) \cdot Dg(x) + Dh(x) && \text{by chain rule} \\
&= (2t \circ g)(x) Dg(x) + Dh(x) && \text{by } D[t^n] = nt^{n-1} \\
&= 2g(x)Dg(x) + Dh(x)
\end{aligned}$$

(b)

$$\begin{aligned}
Df(x) &= D[g(h(x))^2 + h(h(x))] \\
&= D[g(h(x))^2] + D[h(h(x))] && \text{by linearity of differentiation} \\
&= D(t^2 \circ g \circ h)(x) + D(h \circ h)(x) \\
&= (D[t^2] \circ (g \circ h))(x) \cdot D(g \circ h)(x) + (Dh \circ h)(x) \cdot Dh(x) && \text{by chain rule} \\
&= (D[t^2] \circ g \circ h)(x) \cdot (Dg \circ h)(x) \cdot Dh(x) + (Dh \circ h)(x) \cdot Dh(x) && \text{by chain rule} \\
&= (2t \circ g \circ h)(x)(Dg \circ h)(x) Dh(x) + (Dh \circ h)(x) Dh(x) && \text{by } D[t^n] = nt^{n-1} \\
&= 2g(h(x))Dg(h(x))Dh(x) + Dh(h(x))Dh(x)
\end{aligned}$$

(c)

$$\begin{aligned}
Df(x) &= D[g(x^3) + h(\cos(x))] \\
&= D[g(x^3)] + D[h(\cos(x))] && \text{by linearity of differentiation} \\
&= D(g \circ t^3)(x) + D(h \circ \cos)(x) \\
&= (Dg \circ t^3)(x) \cdot D[x^3] + (Dh \circ \cos)(x) \cdot D[\cos(x)] && \text{by chain rule} \\
&= (Dg \circ t^3)(x) \cdot 3x^2 + (Dh \circ \cos)(x) \cdot D[\cos(x)] && \text{by } D[t^n] = nt^{n-1} \\
&= (Dg \circ t^3)(x) \cdot 3x^2 + (Dh \circ \cos)(x) \cdot (-\sin(x)) && \text{by } D[\cos(t)] = -\sin(t) \\
&= 3Dg(x^3)x^2 - Dh(\cos(x))\sin(x)
\end{aligned}$$

(d)

$$\begin{aligned}
Df(x) &= D[h(g(x)^2) + \exp(h(x))] \\
&= D[h(g(x)^2)] + D[\exp(h(x))] && \text{by linearity of differentiation} \\
&= D(h \circ t^2 \circ g)(x) + D(\exp \circ h)(x) \\
&= (Dh \circ (t^2 \circ g))(x) D(t^2 \circ g)(x) + (D\exp \circ h)(x) Dh(x) && \text{by chain rule} \\
&= (Dh \circ t^2 \circ g)(x) D(t^2 \circ g)(x) + (\exp \circ h)(x) Dh(x) && \text{by } D\exp(t) = \exp(t) \\
&= (Dh \circ t^2 \circ g)(x)(2t \circ g)(x) Dg(x) + (\exp \circ h)(x) Dh(x) && D(t^2 \circ g)(x) = (2t \circ g)(x) Dg(x) \\
&= 2Dh(g(x)^2) g(x) Dg(x) + \exp(h(x)) Dh(x)
\end{aligned}$$

□

Exercise 7.2. Each of the following functions has a critical point $x^* = 0$:

(a) $f(x) = 1 - x^2$

(b) $f(x) = x^2 - 1000x^{10}$

(c) $f(x) = x^3 e^{-x}$

(d) $f(x) = x^2 - (\cos(x))^2$

Is $x^* = 0$ a local extremal point of the function or not? If it is an extremal point, is it a minimum or a maximum point of the function? Justify your answers by examining how the derivatives behave around $x = 0$.

Solution.

(a) $f(x) = 1 - x^2$. The derivative is

$$f'(x) = -2x.$$

Because the derivative is positive on $(-\infty, 0)$ and negative on $(0, +\infty)$, the function is strictly increasing to the left of $x^* = 0$ and strictly decreasing to the right of it. Therefore x^* is a local (in this case also global) *maximum* point.

(b) $f(x) = x^2 - 1000x^{10}$. The derivative is

$$f'(x) = 2x - 10000x^9 = (2 - 10000x^8)x.$$

The factor x is positive when $x > 0$. The factor $2 - 10000x^8$ is positive if and only if

$$|x| < a := \left(\tfrac{2}{10000}\right)^{1/8}.$$

Therefore, by the zero product rule

	$(-\infty,-a)$	$-a$	$(-a,0)$	0	$(0,a)$	a	$(a,+\infty)$
x	$-$	$-$	$-$	0	$+$	$+$	$+$
$2-10000x^8$	$-$	0	$+$	$+$	$+$	0	$-$
$f'(x)$	$+$	0	$-$	0	$+$	0	$-$
$f(x)$	$\nearrow$	M	$\searrow$	m	$\nearrow$	M	$\searrow$

the derivative is negative when $x \in (-a,0)$ and positive when $x \in (0,a)$. Thus the function is strictly decreasing to the left of $x^* = 0$ and strictly increasing to the right of it. Therefore x^* is a local *minimum* point. (This function does not have a global minimum point.)

(c) $f(x) = x^3 e^{-x}$. The derivative is

$$f'(x) = 3x^2 e^{-x} + x^3 \cdot (-e^{-x}) = (3-x)x^2 e^{-x}.$$

The factor $x^2 e^{-x}$ is always non-negative, while $3-x$ is positive if $x < 3$ and negative if $x > 3$. Thus the derivative is positive both on $(-\infty,0)$ and $(0,3)$, which means that the function is strictly increasing both to the left and right of $x^* = 0$. Therefore the critical point $x^* = 0$ is *not an extremal point*.

(d) $f(x) = x^2 - (\cos(x))^2$. The derivative is

$$f'(x) = 2x + 2\cos(x)\sin(x) = 2(x + \cos(x)\sin(x)).$$

Recall from trigonometry that $\cos(x) > 0$ if $|x| < \pi/2$ and $\sin(x) > 0$ if $0 < x < \pi/2$ and $\sin(x) < 0$ if $-\pi/2 < x < 0$. This means that $\operatorname{sgn}\big[\cos(x)\sin(x)\big] = \operatorname{sgn}[\sin(x)]$ when $|x| < \pi/2$. Moreover, note that $\operatorname{sgn}[\sin(x)] = \operatorname{sgn}[x]$ when $|x| < \pi/2$. Consequently, $x + \cos(x)\sin(x) < 0$ if $-\pi/2 < x < 0$ and $x + \cos(x)\sin(x) > 0$ if $0 < x < \pi/2$. Thus the function is strictly decreasing to the left of $x^* = 0$ and strictly increasing to the right of it. Therefore x^* is a local *minimum* (in this case also global) point.

Note that sgn is the signum function and it is defined as $\operatorname{sgn}(x) := \begin{cases} -1 & \text{if } x < 0, \\ 0 & \text{if } x = 0, \\ 1 & \text{if } x > 0. \end{cases}$

□

Exercise 7.3. Let $C > 0$ and suppose that f is a differentiable function that satisfies the inequality

$$f(x) - f(y) - f'(y)(x-y) \le C(x-y)^2 \quad \text{for all} \quad x, y \in \mathbb{R}. \tag{1}$$

Let $\eta > 0$ and $x_0 = 0$. Define recursively $x_{k+1} = x_k - \eta f'(x_k)$ for $k \ge 0$.

(a) Show that $f(x_{k+1}) < f(x_k)$ if $f'(x_k) \neq 0$ and $\eta < 1/C$.

(b) Does the function $f(x) = x^2 + 1$ satisfy the inequality in Equation (1) for some $C > 0$? Justify your answer.

Solution.

(a) Setting $x = x_{k+1}$ and $y = x_k$ in Equation (1) yields

$$f(x_{k+1}) - f(x_k) - f'(x_k)(x_{k+1} - x_k) \leq C(x_{k+1} - x_k)^2.$$

This is equivalent to

$$f(x_{k+1}) \leq f(x_k) + f'(x_k)(x_{k+1} - x_k) + C(x_{k+1} - x_k)^2.$$

By the recursion $x_{k+1} = x_k - \eta f'(x_k)$ we have

$$x_{k+1} - x_k = -\eta f'(x_k).$$

Substituting it into the preceding inequality gives

$$\begin{aligned} f(x_{k+1}) &\leq f(x_k) + f'(x_k)\left[-\eta f'(x_k)\right] + C\left[-\eta f'(x_k)\right]^2 \\ &= f(x_k) - \eta f'(x_k)^2 + C\eta^2 f'(x_k)^2 \\ &= f(x_k) + (C\eta - 1)\eta f'(x_k)^2. \end{aligned}$$

The assumption $\eta < 1/C$ implies that $C\eta - 1 < C(1/C) - 1 = 1 - 1 = 0$. That is, $\alpha := 1 - C\eta > 0$. Therefore

$$f(x_{k+1}) \leq f(x_k) + (C\eta - 1)\eta f'(x_k)^2 = f(x_k) - \alpha\eta f'(x_k)^2.$$

Since here $\alpha > 0$, $\eta > 0$ and $f'(x_k)^2 > 0$ by assumption, we get

$$f(x_{k+1}) \leq f(x_k) - \alpha\eta f'(x_k)^2 < f(x_k),$$

which is the desired inequality. This shows that each step in gradient descent decreases the value of f if the function f satisfies Equation (1), which often does not happen.

(b) For $f(x) = x^2 + 1$ we have $f'(x) = 2x$. Therefore

$$\begin{aligned} f(x) - f(y) - f'(y)(x - y) &= (x^2 + 1) - (y^2 + 1) - 2y(x - y) \\ &= x^2 - y^2 - 2xy + 2y^2 \\ &= x^2 - 2xy + y^2 \\ &= (x - y)^2 \end{aligned}$$

Thus Equation (1) is satisfied for any $C \geq 1$.

□

Exercise 7.4. Use automatic differentiation in a programming language of your choice to plot the derivative of the following function on the interval $[0, 1]$:

$$f(x) = \sin(\cos(\sin(\cdots\cos(x)\cdots))) = (\sin \circ \cos \circ \sin \circ \cdots \circ \sin \circ \cos)(x),$$

where there are 50 sines and 50 cosines that alternate. You can either implement automatic differentiation yourself or use an existing library.

Solution. Firstly, let's implement automatic differentiation ourselves in *forward mode* starting from the sample code provided in the lecture notes. Here, it is presented in a stripped down version, where we have kept what is strictly needed to solve the exercise. A more detailed explanation of the underlying principle can be found in the lecture notes.

```python
import numpy as np
import matplotlib.pyplot as plt


# automatic differentiation forward-mode
class fDf:
    def __init__(self, val, deriv):
        self.val = val
        self.deriv = deriv


def sin(u: fDf) -> fDf:
    return fDf(np.sin(u.val), np.cos(u.val) * u.deriv)


def cos(u: fDf) -> fDf:
    return fDf(np.cos(u.val), -np.sin(u.val) * u.deriv)


# f(x) = sin(cos(sin(...cos(x)...))) nested 50 times with a loop
def f(x):
    for _ in range(50):
        x = sin(cos(x))
    return x


# x in [0, 1]
x = np.r_[0:1:101j] # u(x) = x
dx = 1 # u'(x) = 1


# derivative of f at x
dfx = f(fDf(x, dx)).deriv # Df(x)


# plot
plt.plot(x, dfx)
plt.title(r"$Df(x)$")
plt.xlabel(r"$x$")
plt.grid()
plt.show()
```

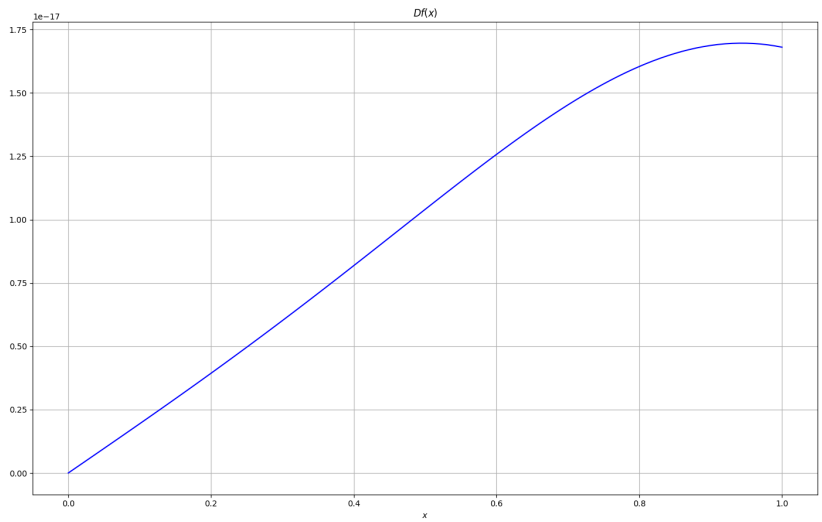
In practice, one would typically use the `JAX` Python library, which provides an efficient (and convenient) implementation of automatic differentiation in *reverse mode* through the `grad` function, of which a thorough explanation of its usage can be found in The Autodiff Cookbook from the documentation. We have reported here a brief introduction for convenience:

> `grad` takes a function and returns a function. If you have a Python function `f` that evaluates the mathematical function f, then `grad(f)` is a Python function that evaluates the mathematical function ∇f. That means `grad(f)(x)` represents the value $\nabla f(x)$.

Note that, `jax.grad` supposes that `f` is a scalar-valued function, and therefore `grad(f)` will only be able to evaluate one point x at a time. To compute $\nabla f(x)$ over a batch of inputs (i.e., an `Array`) we can use the `vmap` function, which returns the vectorised map of a Python function; you can read more about automatic vectorisation in `JAX`'s documentation.

```python
import jax.numpy as jnp
from jax import grad, vmap
import matplotlib.pyplot as plt

def f(x):
    for _ in range(50):
        x = jnp.sin(jnp.cos(x))
    return x

df = vmap(grad(f)) # derivative of f (vectorised)

x = jnp.r_[0:1:101j]
dfx = df(x) # Df(x)

plt.plot(x, dfx)
plt.title(r"$Df(x)$")
plt.xlabel(r"$x$")
plt.grid()
plt.show()
```

`grad` and `vmap` are examples of what `JAX` calls "composable transformations", i.e. operators that can be composed and applied to any functionally pure Python function.



$Df(x)$ (i.e. `dfx`, in the context of our program) and its plot should result the same whether you implement automatic differentiation yourself or use an existing library (for the same input points `x`, of course) since it is exact by definition.

□

Exercise 7.5. Use a programming language of your choice to implement the univariate gradient descent algorithm described in Section 7 of the lecture notes.

Plot the following functions on $[-5,5]$ and indicate their local and global minima:

$$f(x) = x^2 - 8(\cos(x))^2 \quad \text{and} \quad f(x) = x^2 + 2(\sin(x))^2.$$

Apply the algorithm you have implemented to these two functions with parameters $\eta = 0.05$, $t = 10^{-4}$, $n_{\max} = 10^3$, and $x_0 \in \{1,2,3,4,5\}$. For each f and x_0, record

(i) the minimum point returned by your algorithm;
(ii) whether this point is a local or global minimum; and
(iii) the number of times the algorithm evaluated the derivative.

You can compute derivatives by hand or use an automatic differentiation library as described in the lecture notes.

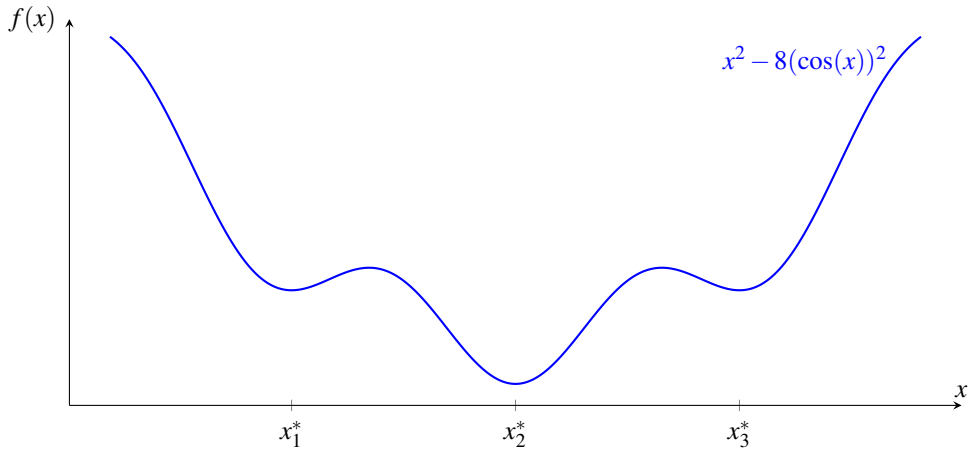
Solution. Again, we will use the superlative `JAX` Python library to perform automatic differentiation in reverse mode. The following code implements the univariate gradient descent algorithm from the lectures notes, where the iteration formula is

$$x_{k+1} = x_k - \eta f'(x_k) \tag{2}$$

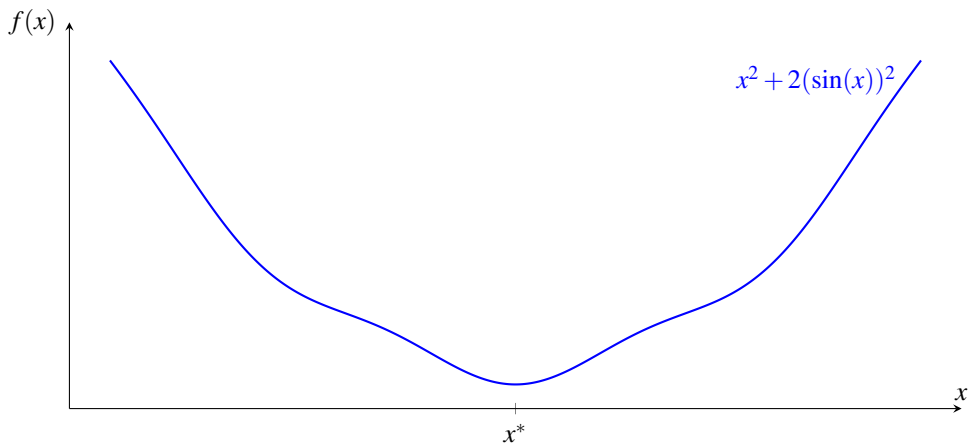
and the loop is terminated after $n_{\max} = 10^3$ iterations or when $\left|f'(x_k)\right| < t = 10^{-4}$.

```
import jax.numpy as jnp
from jax import grad


# univariate gradient descent
def unigd(f, x: float, learnrate=0.05, tol=1e-4, maxsteps=10**3):
    df = grad(f) # autodiff

    # iterate
    for n in range(maxsteps): # exit condition based on max iterations
        if abs(dfx := df(x)) < tol: # exit condition based on precision
            # terminate when f'(x) is small enough
            break
        else:
            # gradient descent step
            x -= learnrate * dfx

    # results
    return x, n


# definitions
x0 = jnp.r_[1.0:6]
f1 = lambda x: x**2 - 8 * jnp.cos(x) ** 2
f2 = lambda x: x**2 + 2 * jnp.sin(x) ** 2


# solve
for f in (f1, f2):
    print()
    for x in x0:
        x_min, n = unigd(f, x)
        print(f'{x=:.16} -> x_{n}={x:.6f}')
```

Now, let's take a closer look at our functions. We have to locate and classify their minima, we can do so by analysing their graphs.



The function $f(x) = x^2 - 8(\cos(x))^2$ has three local minimum points, $x_1^* \approx -2.76072$, $x_2^* = 0$, and $x_3^* \approx 2.76072$. Of these $x_0^* = 0$ is the unique global minimum point.



The function $f(x) = x^2 + 2(\sin(x))^2$ has only one local minimum point, $x^* = 0$. This is also the unique global minimum point.

The table below (on the **left**) gives the results for $f(x) = x^2 - 8(\cos(x))^2$ obtained by the model code for each initial point x_0. Here $x_{\min}$ are the minimum points returned by the gradient descent algorithm, n is the number of derivative evaluations and "type" the type (local or global) of the minimum point $x_{\min}$. For $x_0 \in \{2,3,4,5\}$ the algorithm finds only a local minimum point. However, for $x_0 = 1$ it is able to find the global minimum point.

We have the same table (on the **rigth**) for $f(x) = x^2 + 2(\sin(x))^2$. The algorithm always find the global minimum point, which is not surprising since this is the only critical point that the function has.

$f(x) = x^2 - 8(\cos(x))^2$

x_0	*(i)* $x_{\min}$	*(ii)* type	*(iii)* n
1.0	0.000001	global	7
2.0	2.760719	local	14
3.0	2.760728	local	9
4.0	2.760730	local	10
5.0	2.760727	local	13

$f(x) = x^2 + 2(\sin(x))^2$

x_0	*(i)* $x_{\min}$	*(ii)* type	*(iii)* n
1.0	0.000016	global	32
2.0	0.000014	global	39
3.0	0.000013	global	45
4.0	0.000015	global	47
5.0	0.000015	global	49

Below, we present a *vectorised* version of the univariate gradient descent algorithm that is able to evaluate over a batch of inputs `x0` at once (in a *array programming* fashion).

We cannot use standard Python control flow and logical operators (like `if`, `else`, `break`, `and`, `or`, ...) inside a function that we aim to vectorise. For, `JAX` requires that function is *functionally pure* (see the model solution of Exercise 7.4). Therefore we use `jax.lax.while_loop` (see the documentation for the semantics of `while_loop` in a Python's `itertools` fashion) to rewrite our iteration; `_unigd` is otherwise equivalent to `unigd` from Exercise 7.5. Now, for `JAX`, the function `_unigd` is functionally pure and can be successfully "composed", i.e vectorised or JIT compiled.

We use `jax.vmap` to vectorise `_unigd` along the "input axis" `x0`: the `in_axes` argument specifies position-wise what arguments of `_unigd` should be vectorised, i.e. `None` at indexes 0, 2, 3, 4 tell `jax.vmap` that the arguments at positions 1, 3, 4, 5 of `_unigd` (that are, `f`, `learnrate`, `tol`, and `maxsteps`) are scalars while the `0` at index 1 tells that the second input argument (that is, `x0`) is an `Array` and should be vectorised along the the 0th-axis (that is, in Python indexing, the first dimension of the array). We are constructing the *univariate* gradient descent algorithm therefore it's reasonable to expect that `x0` will be a one-dimensional array where each entry represents a different x_0 in the context of our algorithm (2).

`vectorised_unigd`, being functionally pure, didn't carry over the defaults specified in the variadic definition of `_unigd` (which we kept also there just for convenience in case we wish to use `_unigd` for debug reasons). Therefore, let's wrap `vectorised_unigd` into another function `unigd` that will reimplement our defaults. Finally, we have a handy function `unigd` that is vectorised and thus can operate over a batch of x_0 simultaneously.

```
import jax
import jax.numpy as jnp

jnp.set_printoptions(suppress=True, precision=6)


# constants
ETA = 0.05
T = 1e-4
N_MAX = 1000


# univariate gradient descent
def _unigd(f, x0, learnrate=ETA, tol=T, maxsteps=N_MAX):
    df = jax.grad(f)

    def cond_fun(state):
        x, n = state
        return jnp.logical_and((jnp.abs(df(x)) >= tol), (n < maxsteps))

    def body_fun(state):
        x, n = state
        return x - learnrate * df(x), n + 1

    init_val = (x0, n := 0)
    x_min, *_ = jax.lax.while_loop(cond_fun, body_fun, init_val)

    return x_min


# vectorise along the second input argument
vectorised_unigd = jax.vmap(_unigd, in_axes=(None, 0, None, None, None))
```

```python
# wrapper with variadic definiton (i.e. with defaults)
def unigd(f, x0, learnrate=ETA, tol=T, maxsteps=N_MAX):
    return vectorised_unigd(f, x0, learnrate, tol, maxsteps)


# definitions
x0 = jnp.r_[1.0:6]
f1 = lambda x: x**2 - 8 * jnp.cos(x) ** 2
f2 = lambda x: x**2 + 2 * jnp.sin(x) ** 2


# solve
for f in (f1, f2):
    x_min = unigd(f, x0)
    print(jnp.c_[x0, x_min])
```

The vectorisation yields substantial efficienciy gains: the above model code runs 5 to 10 times faster then the previous version.

□